

# AI specific Storage

## Vector Databases

A. vector DB stores high dimensional vectors (numerical representations of data). Vectors come from text, image, audio embeddings.

- AI ~~can~~ models converts texts, images into embeddings.
- Similar meaning produce similar vectors
- So instead of exact keyword match, we perform nearest neighbor search in vector space, this is similarity search

## Why not normal DB

- Imagine 1 million docs represented as 768-dimensional vector each. ~~to~~ 1

To find similar docs, you must compute distance b/w vectors. Normal SQL db are not optimized for high dimensional similarity search.

## Problems Vector DB solves

- They enable:
- Semantic search
  - AI-powered retrieval
  - Recommendation system
  - RAG systems
  - Similarity matching

## Working

\* Store vector  $\rightarrow$  Index vector  $\rightarrow$  Perform similarity search

Similarity is measured using:-

- Cosine similarity
- Euclidean distance
- Dot product

Instead of:- WHERE value = X

we do: find top 5 vectors closest to this vector.



## Why Indexing?

If you compare query vector with every vector in DB, it is very slow.

- Vector DB use special indexing techniques:-
  - Approximate Nearest Neighbor (ANN)
  - Hierarchical Navigable Small World (HNSW)
  - Inverted file Index (IVF)

Trade offs :-  
→ Reduce search space & huge speed improvement  
→ Slight accuracy loss.

## Real Backend Use case

### 1) RAG (Retrieval-Augmented Generation)

- User question → convert to embedding
- Search vector DB → retrieve relevant documents
- Send to LLM → Generate answers.

### 2) Semantic Search

Search engine that understands meaning, not just keywords.

Ex:- 'Cheap hotels near airport'. (⇒ Match related phrases)

### 3) Recommendation System → Find users with similar behaviour.

### 4) Image search → Upload image → find similar image.

## Examples

- Pine cone
- Weaviate
- Milvus

- Qdrant } specialized vector DB
- Elastic search (with vector support)
- PostgreSQL + pgvector  
→ Extended

## Challenges

- Large memory usage
- High storage for embeddings
- Approximate results
- Index rebuild complexity.
- Scaling across nodes.



## Embedding Storage

- It means persisting AI-generated vector representations so they can be searched later.
- They are usually stored with :-
  - Metadata (doc\_id, user\_id, tags)
  - Original content reference

### Why Proper Storage?

- Poor storage design causes :
  - slow search
  - High memory usage
  - Data inconsistency
  - Expensive queries
  - Difficult scaling

### Design Consideration

1) Metadata storage :- Embeddings alone are not enough.

We must store :- i) Doc\_id ii) user\_id iii) Access control info

Because after similarity search, we fetch ~~not~~ real content using metadata.

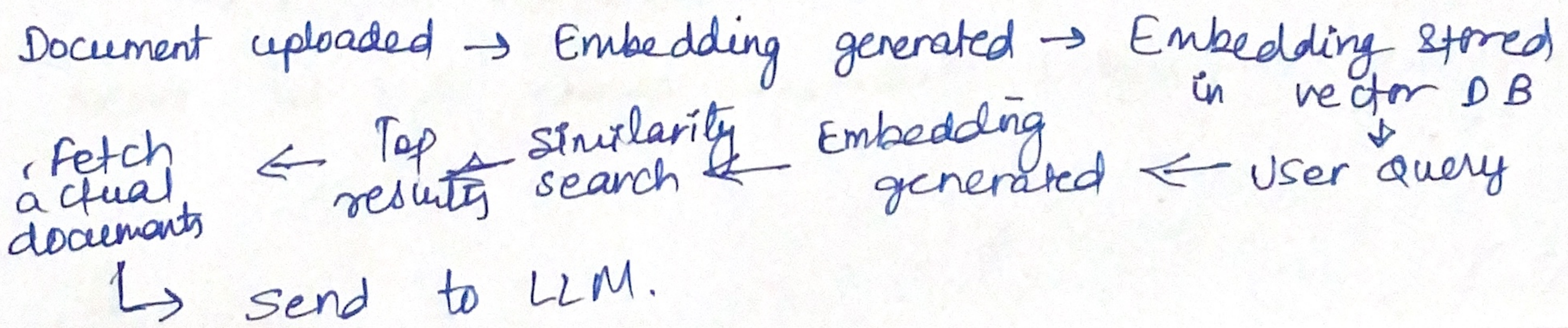
2) Multi-Tenant Isolation :- In SaaS system, User A should not retrieve User B's data. Embedding storage must support:

- Tenant level filtering
- Metadata based search constraints

3) When document changes → do we regenerate embeddings and update index immediately?  
↓  
Design carefully

4) Hybrid search :- Often combine: Keyword & vector search  
Why? → vector search finds meaning, Keyword search ensures precision.

### Architecture





## Metadata filtering

Ex:- Suppose we have AI document assistant.

- 100 companies, each uploads document and all embeddings are stored in same vector DB.

Now, user from company A asks → 'Show my HR policy'  
vector search may return → 'HR policy from company B'.

So, we need a way to filter results before or during similarity search.

- Metadata filtering means restricting vector search results based on structured attributes. Each embedding is stored with metadata like:

• document\_id • tenant\_id • user\_id • timestamp.

## Filtering Strategies

1) Pre-filter then search → Filter docs by metadata, then perform similarity search.

- works well if filter reduces datasets significantly.

2) Filter during search

- Applying metadata constraints inside ANN search.
- More efficient at scale.

## Architecture

\*) User - Backend

- Extract tenant\_id
- Generate embedding
- Search with enforced metadata filter
- Validate result
- Return response

Never let LLM choose metadata, filters.



## Hybrid search

- \* User searches → 'Invoice number 45721'
- Vector search may focus on 'invoice' meaning but may not precisely match exact no. 45721.
- \* Keyword search is better for exact matches.
- Hybrid search combines both semantic relevance and lexical relevance (exact match).

### Working

- 1) User query comes → System performs: vector similarity search + keyword search (BM25 or inverted index).
- 2) Combine results & rank results using weighted scoring

Ex!  $\boxed{\text{Final Score} = 0.7 * \text{vector\_score} + 0.3 * \text{keyword\_score}}$

### Hybrid search use case

- AI document assistant
- Legal document search
- Financial search platforms
- E-commerce search

- \* In RAG Systems:  
Bad retrieval = bad generation.

Hybrid search improves:

- Retrieval precision
- Context quality
- LLM answer accuracy



## RAG Document versioning

- In real system, documents change such as company updates policy, Invoices modified, contract revised.
- If we: generate embedding once & never update it. Our AI may retrieve outdated info.

\* RAG document versioning means :- Tracking and managing different versions of documents used in embedding storage.

- \* Each doc & version may have:

1) version\_id      2) created\_at      3) updated\_at  
4) is\_active flag.

- \* When document changes

Option A :- Delete old embedding → Insert new embedding.

Option B :- Make old version inactive → Insert new version  
(Safer)      filter only active version ←  
                                         in search.

## Architecture Flow

- 1) Document Uploaded → Generate embedding
- 2) Store vector + metadata (version = 1, active = true)
- 3) If updated:
- 3) Set version = 1 active = false
- 4) Insert version = 2 active = true
- 5) Generate new embedding

Problem:- Storage increases, index rebuild may be required